SQL_TEST_01 · BUILD 6 · HANDOFF BRIEF

Front End for the Agent

A standalone, deep-dive reference for Build 6 — a Streamlit chat interface wrapping the existing tool-use agent, so the whole project is demoable in a browser instead of only via a Python REPL. Every phase, every real bug the process surfaced, and every verification result, in full. Complements (does not replace) the living cheat sheet and the per-phase flow diagrams.

STATUS

Complete — all phases including 3.5, live-verified in a real browser

COMMITS

`b902500` → `edc84d2`

CONTINUES

Build 5 (live deployment with CI)

PRECEDES

Build 7 (verifier/critic agent)

What Was Built, and Why

Every build through Build 5 deepened the agent or its infrastructure; none of them changed how a person actually reaches it. `agent.py` was still a `print()`-based CLI loop — technically complete, entirely undemoable to anyone unwilling to clone the repo and run Python locally. Build 6 closes that gap: a real, clickable chat interface, wrapping `ask_agent()` as-is, with no changes to the agent's own reasoning.

Five units of work, in order built (not always in number order): a minimal chat UI (Phase 1); genuine multi-turn conversation memory, since nothing before this build ever needed it (Phase 2); containerizing the whole thing as a second service sharing Build 1's own Docker image (Phase 3); a stretch phase surfacing Build 4's tool-call logging directly in the browser (Phase 4); and persistent, restart-proof chat history (Phase 3.5) - identified mid-build from a real live-testing gap, initially deferred, then fast-tracked once a UI refinement made it worth building after all. See Phase 3.5's own section for the full story.

> **Nothing about the agent's own reasoning changed in this build.** Every phase here is front-end/infrastructure work around the existing `ask_agent()` tool-use loop — the one exception is a handful of real, evidence-based fixes to `tools.py`/`agent.py` that live-testing through the new UI surfaced (a missing result cap, stale discovery scope, a stale system prompt) — see Bugs Found & Fixed.

Tech Stack Decision

Decided before any code was written. Streamlit, over two real alternatives.

OPTION	VERDICT
Streamlit (chosen)	Pure Python — no new language or build tooling. Built-in chat primitives (`st.chat_message`, `st.chat_input`) purpose-built for exactly this shape of app. Deployable as another `docker-compose.yml` service, or free via Streamlit Community Cloud.
Flask/FastAPI + a separate JS/React front end	Deferred — more production-shaped in theory, but real scope creep: a second codebase and build-tooling stack for a project whose point is the SQL/data-agent work, not front-end engineering.
Gradio	Deferred — comparable fit, more ML-demo-oriented. Streamlit's broader general adoption and pure-Python fit won out.

1 Minimal Chat UI

Goal: wrap the existing `ask_agent()` loop in a real chat interface — nothing more.

```
app.py — the shape of every later phase builds on

import streamlit as st
from agent import ask_agent

# Streamlit reruns this whole script on every interaction, so history has to live in
# session_state explicitly - a plain Python list would reset to empty on every rerun.
if "messages" not in st.session_state:
    st.session_state.messages = []

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

user_question = st.chat_input("Ask about the database...")
if user_question:
    # append user message, call ask_agent(), append assistant message
```

Why `agent.py` needed a refactor first, not just `app.py`. The original agent was a `__main__`-guarded script, not an importable function. Build 4 had already refactored it into a callable `ask_agent(user_question)` for the eval harness's benefit — this build is the second, independent payoff of that same refactor, reusing it entirely unchanged rather than duplicating agent logic inside the UI layer.

2 Hybrid Tiered Conversation Memory

Goal: real multi-turn memory. Phase 1's `session_state.messages` only gave the *display* history — `ask_agent()` itself still built a fresh one-message list on every call, so the model had zero actual memory of prior turns despite the transcript rendering fine.

Two designs were weighed, not defaulted into. Option A: full raw Anthropic message-block replay (real tool-call memory, but unbounded per-conversation growth). Option B: text-only history (just prior question/answer text pairs). B was initially recommended as the better-scoped fit — then explicitly overridden once the project's own "production-grade portfolio" goal was invoked: real production systems bound/manage history rather than replay it forever (this assistant's own context-compaction being the concrete example), which argues for *B plus an explicit cap*, not for unbounded A. The chosen design went further still, deliberately: **hybrid tiered memory** — recent rounds keep full raw tool-call fidelity, older rounds collapse to plain text.

```
agent.py — the actual mechanism

MAX_FULL_FIDELITY_ROUNDS = 3

def flatten_history(history_rounds):
    older_rounds = history_rounds[:-MAX_FULL_FIDELITY_ROUNDS]
    recent_rounds = history_rounds[-MAX_FULL_FIDELITY_ROUNDS:]

    messages = []
    for round in older_rounds:
        messages.append({"role": "user", "content": round["user_question"]})
        messages.append({"role": "assistant", "content": round["answer_text"]})
    for round in recent_rounds:
        messages.extend(round["full_messages"])
    return messages

def ask_agent(user_question, max_tool_turns=MAX_TOOL_TURNS, history_rounds=None):
    messages = flatten_history(history_rounds or [])
    messages.append({"role": "user", "content": user_question})
    # ...turn loop unchanged... every return now also hands back full_messages
```

A genuine cross-layer nuance, corrected while designing: Streamlit's `session_state` keeps live Python objects in server memory across reruns within one session — it does *not* require JSON serialization the way real cross-process persistence would. Storing raw Anthropic SDK objects (`TextBlock`/`ToolUseBlock`) in `session_state` is simpler than it first sounds; the real complexity is the flatten/collapse logic itself and the API's strict tool_use→tool_result adjacency rule at the fidelity-window boundary.

NOTE-WORTHY — a performance/best-practice analysis was requested and given before any code was written, flagged explicitly for this brief.

Scale tradeoffs identified, none yet mitigated in the design as built:

1. The fidelity window bounds *round count*, not *per-round payload size* — a much larger table means a single retained `run_sql_query` result could be large regardless of how few rounds are kept raw. (This specific gap became real days later — see Bugs Found & Fixed.)
2. A sliding full→collapsed boundary invalidates the API's prompt-cache prefix every turn (the boundary round's content changes each time) — unlike infrequent/periodic compaction (this assistant's own context management, LangChain's `ConversationSummaryBufferMemory`), which preserves a stable cacheable prefix far longer.
3. Vector-search latency (`search_summaries/search_docs`) compounds with conversation-memory latency at real scale.
4. Per-session server memory footprint (N full rounds × concurrent users, once this build containerizes the app) is real concurrency territory — the same category Build 4's `question_id` field exists for.

Best-practice verdict, precise, not a blanket yes/no: the general shape (recent-full + older-degraded) is a real, recognized pattern — it matches LangChain's summary-buffer memory and this assistant's own context compaction. But this project's version is a simplification in two specific ways: it retains old rounds' verbatim text rather than re-summarizing via a dedicated LLM call, and it's recency-based rather than relevance-based (semantic retrieval of the most pertinent prior rounds) — this codebase is unusually well-positioned to do the latter, since pgvector is already proven via `search_summaries/search_docs`. A second, more sophisticated alternative already sits in this codebase too: push large tool outputs into `agent_scratch` instead of retaining them in context, and have the agent re-query rather than re-read.

Disposition: named honestly as future-examination material, not blocking — Phase 2 proceeded on the hybrid design as scoped.

Step 8 — the boundary test, confirmed genuinely, not assumed. Two test designs were tried and honestly discarded before a third worked: the first two were inconclusive because the seed question's own answer was already a complete data dump, so collapsed text alone would have sufficed regardless of whether true collapse was actually exercised. The third design deliberately asked only for an aggregate total up front, forcing the later follow-up to require fresh tool calls if memory genuinely didn't retain the detail.

```
Sample — the confirming 3-question chain, MAX_FULL_FIDELITY_ROUNDS temporarily set to 1

Q1: "What's the average allocation duration, in hours?"          # aggregate only, no row detail
    -> 230.06 hours

Q2: "Which allocation had the most items checked out?"           # unrelated, just advances the
round count
    -> allocation 9993490, 95 items
```

```
Q3: "What were the exact items in that allocation?"              # Q2 is now collapsed, not full-fidelity
    -> correctly re-ran run_sql_query rather than fabricating an answer
    -> allocation 9993490, 6 items # (reconstructed correctly - re-queried, not hallucinated)
```

Reproduced identically across two script-based runs and one real browser session — the system recovers gracefully via re-querying when retained memory doesn't have what's needed, arguably a more important property than "recalls old raw data verbatim."

Bugs Found & Fixed

All five surfaced through genuine live-testing of the new UI, not speculative review — the first real end-to-end usage this agent had ever gotten outside a controlled script or eval case.

1. `run_sql_query` had no result-size cap — broke a live browser question with a real 400 API error before Phase 2's own code was even finished. An unfiltered follow-up-style question (`SELECT ... FROM clean.allocations WHERE actual_end IS NOT NULL ORDER BY actual_end DESC`, no `LIMIT`) matched nearly the whole table and returned 936,551 characters in one tool result — `Error code: 400 ... prompt is too long: 448866 tokens > 200000 maximum`. Every prior use of this tool (Build 1 through the eval harness) happened to involve small tables, aggregates, or model-written `LIMIT`s, so this path had simply never been exercised before.

Fixed: `MAX_SQL_RESULT_ROWS = 200`, `cur.fetchmany(201)` instead of `fetchall()` — fetching one row past the cap makes truncation detectable without a second round-trip. A truncated result gets a trailing `{"message": "Result truncated at 200 rows..."}` sentinel entry, the same pattern `search_summaries`/`search_docs` already use for "no match." This exact gap had already been named as a real, unaddressed risk during Phase 2's own scale discussion above — this incident is that flagged risk surfacing for real, independently confirming the flag wasn't paranoid.

2. The tool-call log never recorded the actual question asked. `logs/tool_calls.jsonl` had every field describing what a tool did, but not what the user had asked that triggered it — discovered while trying to hand back the exact original question that had caused bug 1, and finding it structurally impossible to reconstruct from the log alone.

Fixed: `log_tool_call()` gained a `user_question` parameter, now passed from both of `ask_agent()`'s call sites.

3. `docs.chunks` — the documentation-RAG corpus — had silently stopped growing after Build 3. Found by directly noticing that questions about build history only ever surfaced Build 1–3 content. Root cause: `extract_doc_chunks.py`'s `SOURCE_FILES` is a hardcoded list, written during Build 3.5 and never updated as Build 3.5, 4, and 5 each got their own handoff brief afterward.

Fixed: the three missing brief paths added, extraction/embedding scripts re-run — both already idempotent by design (`DELETE ... WHERE source_file = %s` before re-insert; `WHERE embedding IS NULL` backfill), so

nothing needed to change to make the re-run safe. All 10 source files now present, 395 chunks, 0 unembedded.

4. The log recorded tool activity but never the model's actual final answer. Made correctness impossible to verify from the log alone — had to cross-check every answer against the live browser by hand.

Fixed: a new `log_final_answer(question_id, user_question, answer, error)` event — a distinct shape from tool-call entries (no `tool_name/turn/input/output/latency_ms`), wired into all three of `ask_agent()`'s return points.

5. The system prompt still described `appdb` as a generic "small business database," leftover Build 1 framing. Asking "What can you do?" through the real UI produced example questions like "customers with orders over $500" — confirmed via the log that zero tool calls fired, meaning this was pure stale-framing filler text, not real schema discovery. A real portfolio viewer trying one of those self-suggested examples would have gotten a confusing empty result, since those tables were never the project's actual subject.

Fixed, alongside related housekeeping: the prompt's opening rewritten to describe the actual domain (equipment allocations, `clean.allocations` as the real subject table). Separately, `customers/orders` (original Build 1 seed tables, unrelated to the portfolio narrative since Build 2) removed from `list_tables/describe_table`'s discovery scope — a discovery-level fix, not a hard DB-level lockout, since this isn't sensitive data, just narrative clutter.

6. The same "docs.chunks stopped growing" bug from earlier in this build recurred — this time on this very brief, found while explicitly auditing whether the agent's own documentation knowledge was current through Build 6. `extract_doc_chunks.py`'s `SOURCE_FILES` is a hardcoded list; it had already been patched once this build (Bug 3) to add three missing briefs, but the fix was itself just another manual entry — this brief was written afterward and never added, so `search_docs`'s knowledge of the project's own history was one build stale again, the same day it happened, the same root cause both times.

Fixed at the root this time, not patched again: `SOURCE_FILES` now auto-discovers every file matching `PROJECT_DIAGRAMS/**/*handoff-brief*.html` via `glob`, rather than a hand-maintained list — present and future handoff briefs are included automatically, with no manual step to forget. The corpus was then rebuilt in full (all 9 handoff briefs plus the cheat sheet and project overview, including this brief's own just-updated Phase 3.5 content) and verified with a real `search_docs` query confirming Build 6/Phase 3.5 content is actually retrievable, not just present in the table.

3 Containerization

Goal: run the Streamlit app as a real `docker-compose` service, matching how the CLI agent has run since Build 1.

A real pre-existing bug found before any Phase 3 code was written. `Dockerfile`'s `COPY agent.py tools.py ./` line had never included `logging_utils.py` (added in Build 4) — the `agent` service's image would have failed to build with an `ImportError` the moment it was ever rebuilt, unrelated to Phase 3 itself but only caught because this phase meant touching this exact file.

Fixed: `COPY agent.py tools.py logging_utils.py app.py ./`.

```
docker-compose.yml — one image, two services, differentiated only by command

  agent:
    build: .
    # default CMD ["python", "agent.py"] from the Dockerfile

  streamlit_app:
    build: .
    depends_on: [postgres]
    ports:
      - "8501:8501"
    volumes:
      - ./logs:/app/logs
    command: ["streamlit", "run", "app.py", "--server.address=0.0.0.0"]
```

A genuine Streamlit-in-Docker gotcha: Streamlit defaults to binding `localhost` only — unreachable through Docker's port mapping without `--server.address=0.0.0.0`, which was not obvious from the plain `streamlit run` command used locally in Phase 1.

Stricter than the local dev-server case: Docker's `COPY` bakes source in at build time, with no hot-reload path at all. Editing `agent.py`/`tools.py`/`app.py` on the host has zero effect on an already-running container. Confirmed directly (not assumed) after the Bug 5 system-prompt fix: a `docker exec ... grep` against the running container's own copy of `agent.py` showed the old text still present until a full `docker compose build` + up (recreate) was run.

4 Tools Used Expander

Goal (stretch): surface Build 4's existing tool-call logging directly in the browser, rather than leaving it CLI/log-file-only.

Design: reuse the log as the source of truth, don't track calls twice. Matches the precedent already set by `run_eval.py` (which reads `logs/tool_calls.jsonl` after each `ask_agent()` call rather than tracking separately).

```
logging_utils.py — the one new function this phase needed

def get_tool_calls(question_id):
    if not os.path.exists(LOG_PATH):
        return []
    calls = []
    with open(LOG_PATH) as f:
        for line in f:
            entry = json.loads(line)
            if entry.get("question_id") == question_id and "tool_name" in entry:
                calls.append({"tool_name": entry["tool_name"], "input": entry["input"],
"latency_ms": entry["latency_ms"]})
    return calls
```

`ask_agent()`'s three return points now hand back `question_id` (already generated internally, just never returned before); `app.py` calls `get_tool_calls(question_id)` after each response and renders an `st.expander`, storing the result per round in `session_state` so it survives future reruns instead of only showing once.

> **Deliberate scope limit: input only, never output.** A tool's `output` can be a full row dump (the exact shape Bug 1 above had to be capped for) — showing it in the expander would just move the doom-scrolling problem into a different widget. Tool name, input, and latency (as a small caption) are shown; the underlying data is not.

> **A real follow-up caught live, not during initial build:** the expander was originally hidden entirely when a question triggered zero tool calls (e.g. "What can you do?", answered purely from the system prompt). Confirmed via the log this was correct behavior, not a bug — then changed anyway, per explicit request, to always render `Tools used (0)` with a short explanatory caption, so the UI element is consistently present rather than silently absent.

5 Persistent Chat History (Phase 3.5)

Goal: survive what Phase 3 revealed live — a container restart silently wiping every session's in-memory conversation, with no in-UI signal it had happened. Numbered 3.5 because it was originally scoped as an insertion between Phases 3 and 4; built last, once a UI refinement (below) made it worth fast-tracking rather than leaving deferred.

Storage: one shared global feed (Option B), not per-user/per-session. New table `agent_scratch.chat_rounds` reuses the existing `appdb_agent_writer` role rather than standing up a third one — the same least-privilege philosophy as `save_dataframe`, even though this particular write is application code running on every new round, not an LLM-driven tool call.

```
tools.py — the two new functions this phase needed

def _jsonable(obj):
    # Anthropic SDK response blocks (TextBlock/ToolUseBlock) are Pydantic objects embedded
    # in assistant turns - not JSON-serializable directly. json.dumps calls this for
    # anything it can't handle itself.
    if hasattr(obj, "model_dump"):
        return obj.model_dump()
    raise TypeError(...)

def save_chat_round(question_id, user_question, answer_text, full_messages):
    # ...connects as appdb_agent_writer...
    cur.execute(
        "INSERT INTO agent_scratch.chat_rounds (...) VALUES (%s, %s, %s, %s::jsonb)",
        (question_id, user_question, answer_text, json.dumps(full_messages, default=_jsonable)),
    )

def load_chat_rounds():
    # ...connects as appdb_reader, ORDER BY round_id ASC...
    # psycopg deserializes jsonb natively - full_messages comes back as plain dicts,
    # the same shape flatten_history() already expects from a live session.
```

> **The serialization wrinkle flagged at design time resolved exactly as predicted, with zero changes to `flatten_history()` itself.** The one real complexity was getting SDK objects into and back out of JSONB correctly — once that round-trips, persistence is invisible to the memory-threading logic, which just sees plain dicts either way.

A UI refinement, proposed after this phase was already scoped and deferred, is what made fast-tracking it worthwhile — and it resolved two open gaps from the original design at once. Rather than per-user attribution tags or a hard cutoff on how much history renders, only the single most-recent round shows expanded (full chat bubbles); every earlier round collapses into `st.expander(label`

= the question text itself). This clears up "which answer is mine" differently than originally scoped — not by tagging identity, but by guaranteeing whoever just asked a question always sees their own answer as the one left open — and solves the display-length concern better than a hard last-N cutoff would have: nothing is ever hidden, it just collapses to a one-line label, so the feed can grow indefinitely without the rendered page growing with it. A genuine side effect: the collapsed labels read as a scannable, running FAQ.

> **One real Streamlit constraint hit during the build, not before: expanders can't nest.** Phase 4 already puts a `Tools used` expander under every answer — so a collapsed historical round can't wrap that same widget inside the new outer FAQ-expander. Resolved by parameterizing the render helper: inside a collapsed round, tool-call detail renders as plain inline text/captions instead of a second expander; the one currently-expanded round (never itself inside an expander) keeps the real collapsible widget.

Verified against the actual failure mode this feature exists to prevent, not a looser proxy for it. A genuine two-process test: one Python process asked a real question and called `save_chat_round()`, then exited completely; a second, entirely separate process — zero shared memory, zero shared state — called `load_chat_rounds()` and correctly recalled the prior answer with no re-query. Confirmed for real afterward too: an actual `docker compose restart` preserved history visibly in the browser, not just this script-level proxy for it.

> **A real, previously-only-predicted limitation, confirmed directly via a two-tab test.** Asking a question in one open tab doesn't appear in a second already-open tab until that tab is refreshed or asks its own question. Not a bug — Streamlit only reruns a given tab's script on that tab's own interaction; each tab holds its own independent `session_state`, loaded from the database once at that tab's first load, with no push mechanism to other open sessions. This is exactly the "not live-synced across tabs" tradeoff named when Option B was first chosen, now empirically confirmed rather than just anticipated.

Follow-up fix, same day: no cap or cleanup existed on either the displayed feed or the underlying table, until asked directly. Neither `load_chat_rounds()` nor the render loop had a `LIMIT` anywhere — every round ever saved would be loaded into every session and rendered (collapsed, but still rendered) forever, and `agent_scratch.chat_rounds` had no rotation mechanism at all, unlike `logs/tool_calls.jsonl`'s own `LOG_ROTATION_THRESHOLD_LINES` from a Build 4 follow-up.

```
tools.py — trim on write, same spirit as the log's own rotation
MAX_CHAT_ROUNDS = 200  # arbitrary safety cap, not a real capacity limit

def save_chat_round(question_id, user_question, answer_text, full_messages):
    # ...insert the new round...
    cur.execute(
```

```
        """
        DELETE FROM agent_scratch.chat_rounds
        WHERE round_id <= (
            SELECT round_id FROM agent_scratch.chat_rounds
            ORDER BY round_id DESC OFFSET %s LIMIT 1
        )
        """,
        (MAX_CHAT_ROUNDS,),
    )
```

The subquery doubles as its own no-op guard. Below the cap, `OFFSET` runs past the end of the table and the subquery returns `NULL` — `round_id <= NULL` is never true in SQL, so nothing gets deleted, with no separate count-then-branch check needed the way `save_dataframe`'s `MAX_SCRATCH_TABLES` check requires.

A real gap in the original migration, found immediately by running this against the live database, not by review. `012_chat_rounds_schema.sql` had only ever granted `SELECT, INSERT` to `appdb_agent_writer` — never `DELETE`, since the original design had nothing that deleted anything. The new trim step failed with `InsufficientPrivilege` on the very first test run. Fixed by granting `DELETE` live against `pg_local`, then amending `012_chat_rounds_schema.sql` itself in place (the same precedent as Build 5's `CREATE EXTENSION vector` fix to `008_add_summary_embedding.sql`) so a genuinely fresh database gets the correct grant from one file, not a follow-up migration.

Verified with a real boundary test (`test_chat_rounds.py`, new): `MAX_CHAT_ROUNDS` monkeypatched to `3`, 5 real rounds saved, confirmed only the 3 most recent survive and the 2 oldest are gone - plus a save/load round-trip test and a discovery-exclusion test, 3 tests total. Full suite 25/25 (up from 22).

Design Discussion

One Image, Two Containers

Two services, two separate needs, not a redundancy. `agent` (the CLI container) is the original Build 1 deliverable, proving the core tool-use loop works, containerized and reproducible — also what Build 5's CI actually exercises (migrations + pytest run against the real agent code), the engineering foundation everything else sits on. `streamlit_app` (the web container) was built specifically because the CLI isn't demoable to a portfolio reviewer — per Build 6's own original scoping: a real, clickable interface so the project is demoable without cloning the repo and running Python locally.

Both containers are built from the **same image** (one `Dockerfile`, one `requirements.txt`, one set of copied source files) but run with **different commands** — not two separate images, not one container doing double duty. If asked "which one is the final project": the Streamlit container is what a recruiter would actually click through, but the CLI/agent container isn't superseded by it — it's the tested, CI-verified core the Streamlit app is a front end on top of. Both stay in the final project, serving different audiences.

What Is `sentence-transformers` Actually Doing?

Not solely focused on the allocations data — it plays the same *mechanical* role in two entirely separate places. It converts text into a fixed-length numeric vector (384 dimensions, `all-MiniLM-L6-v2`) — nothing more. It never writes any part of the agent's actual response; that's entirely Claude's job. The embedding model's only output is "here's a vector representing the meaning of this text," compared to pre-computed vectors via distance (`<=>` in pgvector) to find semantically similar rows.

Two distinct uses in `tools.py`, sharing one cached model instance: `search_summaries()` embeds the user's query and compares it against `clean.allocations.summary_embedding` — the allocations data itself. `search_docs()` embeds the query and compares it against `docs.chunks.embedding` — this project's own build history and design decisions, nothing to do with the allocations database at all.

The actual flow, end to end: the embedding model finds candidate rows semantically close to the question. Those rows return to Claude as a normal tool result, the same way `run_sql_query`'s rows would — and Claude is the one that reads them and writes the actual prose answer. `sentence-transformers` never sees the conversation, never generates text for the user, and has no awareness of which tool called it.

Known Limitations & Open Items

Resolved this build: chat history is now persistent — see Phase 5 (Phase 3.5) above. A real container restart no longer wipes conversation memory; a `PROCESS_ID` diagnostic (generated once per process, shown as a UI caption) still makes a restart visible when one does happen.

Not live-synced across open tabs — confirmed via real two-tab testing, not just anticipated at design time. See Phase 3.5's write-up above for the root cause and why it's a deliberate simplicity tradeoff of the shared-feed design, not a defect. True live push across tabs would need additional infrastructure (polling, or Streamlit's newer auto-refresh primitives) not built in this pass.

Cosmetic-only: Streamlit's dev-mode file watcher logs a harmless `ModuleNotFoundError: No module named 'torchvision'`. Root-caused to the watcher probing optional vision-model submodules while walking `sentence-transformers`'s dependency tree — this project never installs or needs `torchvision`. Zero functional impact, confirmed every time it came up; a `[logger] level = "error"` config fix was attempted and could not be confirmed effective in real use, so it was removed rather than kept as unverified.

Verification Log

What was actually run against a real environment or a real browser, not just read for plausibility.

Phase 1
Live-verified in the browser; independently rebuilt in place later as a comprehension check before Build 6 was considered closed.

Phase 2, boundary test
Reproduced identically across two script-based runs and one real browser session (see the sample transcript above).

Each of the 6 bugs above
`py_compile` on every changed file; a direct functional reproduction of the original failure against the fixed code; full pytest suite re-run after every single fix (19 → 22 passing across the build).

Phase 3
Both images built, stack brought up, live-verified in the browser with log persistence confirmed via the host filesystem bind mount; a stale unrelated container from an earlier build cleaned up so only the current one remains.

Phase 4
`py_compile` clean; a direct functional test of `get_tool_calls` (temp log file — correct `question_id` filtering, `output` excluded, unknown-ID returns `[]`); full pytest suite 22/22; a real live `ask_agent()` call confirming `question_id` threads end-to-end to real logged data; confirmed live in the browser for both the zero-tool-call and multi-tool-call cases.

Phase 3.5
`py_compile` clean; a save/load round-trip test against the live table (JSONB serialization survives intact, discovery exclusion confirmed); full pytest suite 22/22; a genuine two-process restart-simulation test (one process saves and exits, a second unrelated process loads from the DB only and correctly recalls the answer); container rebuilt and confirmed via `docker exec ... grep` to actually be running the new code; then confirmed for real by the user — an actual container restart, a two-tab shared-feed check, and the FAQ-expander's look and feel, all in a live browser.

Sample — live browser confirmation, Phase 4's zero-tool-call case

```
Q: "What can you do?"
-> answered entirely from the system prompt, zero tool calls
-> Tools used (0)
   "Answered directly - no tools were called for this question."

Q: "Tell me about the database?"
-> list_tables {} 22ms
```

```
-> describe_table {"table_name": "allocations"} 20ms
-> Tools used (2), both persisted correctly after the next question was asked
```

Environment State

COMPONENT	DETAIL
New Python	app.py — the Streamlit entrypoint
Changed Python	agent.py (ask_agent() callable refactor carried in from Build 4, flatten_history(), MAX_FULL_FIDELITY_ROUNDS, PROCESS_ID, question_id returned, SYSTEM_PROMPT rewritten), tools.py (MAX_SQL_RESULT_ROWS cap, customers/orders + chat_rounds removed from discovery, save_chat_round()/load_chat_rounds(), MAX_CHAT_ROUNDS retention cap), logging_utils.py (user_question field, log_final_answer(), get_tool_calls()), extract_doc_chunks.py (SOURCE_FILES switched from a hand-maintained list to glob auto-discovery, see Bugs Found & Fixed)
New SQL	012_chat_rounds_schema.sql — agent_scratch.chat_rounds, applied live against pg_local
Changed infra	Dockerfile (COPY fixed to include logging_utils.py/app.py), docker-compose.yml (new streamlit_app service), requirements.txt (streamlit added)
New tests	test_agent_history.py — 3 tests for flatten_history(); test_chat_rounds.py — 3 tests (round-trip, discovery exclusion, MAX_CHAT_ROUNDS boundary)
Diagram folder	PROJECT_DIAGRAMS/BUILD_6_FLOW/ — phase1/phase2/phase3/phase3-5/phase4 flow diagrams, this brief.
Live app	http://localhost:8501 (streamlit_app service) — unreachable without --server.address=0.0.0.0 set on the container, see Phase 3
Project test suite	25/25 passing (up from 19 at the start of this build)

Build 6 complete — every phase including 3.5, live-verified in a real browser (real restart survives, two-tab shared feed confirmed, FAQ-expander confirmed). Next: Build 7 (a verifier/critic agent). Ongoing cross-build command reference lives in `sql-test-01-cheatsheet.html`.